

Упражнение 6 ЕФП – 2025-2026

Задача 1. Създайте вашата първа програма „hello world“.

1. От меню **File** изберете **New Text File**



2. Запишете следния код във файла:

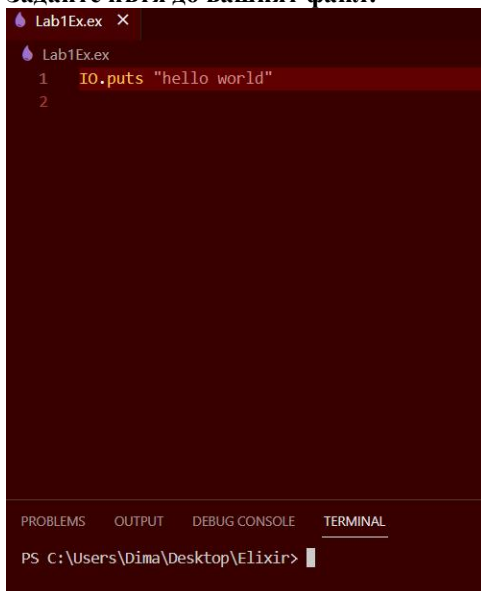
```
IO.puts "hello world"
```

3. Запазете файла с име: **Lab1Ex**. Разширението се добавя автоматично *.ex

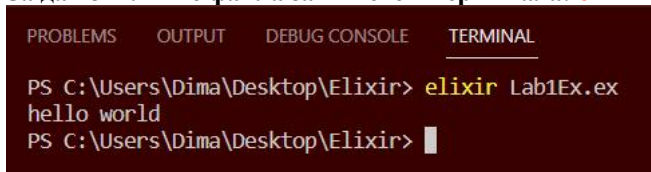
4. От меню **Terminal** изберете:



5. Задайте пътя до вашият файл:



6. За да изпълните файла запишете в терминала: **elixir името_на_файла.ex**



Задача 2. Създайте вашата първа програма, като използвате интерактивния режим за работа. За целта **запишете в терминала - iex.bat**, ако сте на Windows.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS C:\Users\Dima\Desktop\Elixir> iex.bat
Interactive Elixir (1.13.2) - press Ctrl+C to exit (type h() ENTER for help)
iex(1)> █
```

А изход от този режим **Ctrl+C**

```
PS C:\Users\Dima\Desktop\Elixir> iex.bat
Interactive Elixir (1.13.2) - press Ctrl+C to exit (type h() ENTER for help)
iex(1)> 40+2
42
iex(2)> Terminate batch job (Y/N)? █
```

Задача 3. Проектът използва Elixir и Ecto за създаване на система за управление на студенти с база данни SQLite. Основният модул Student дефинира схемата на таблицата с полета fname, mname, lname, fnumber и mark. Връзката с базата данни се осъществява чрез модул Repo, който използва адаптера Ecto.Adapters.SQLite. Конфигурацията на проекта определя файла на базата данни students.db, размера на пул за връзки и режима на журнала. Миграцията 001_create_students.exs създава таблицата students и уникален индекс за полето fnumber. Модулът Logic съдържа CRUD операции за работа с базата: добавяне, показване, изтриване и актуализиране на студенти. Добавянето на студент се реализира чрез add_student/1, която използва changeset за валидация и след това записва в базата. Валидацията включва проверка за присъствие на задължителни полета и уникалност на fnumber. Извличането на всички студенти се реализира чрез list_students/0, като записите се сортират по fnumber. Изтриването на студент се осъществява чрез delete_student/1, който намира студента по fnumber и го премахва от базата. Обновяването на данни се прави чрез update_student/2, която намира целевия запис и прилага промяната чрез changeset. Всички операции с базата се извършват транзакционно и гарантират консистентност на данните. При стартиране на приложението се извиква Application.start/2, който стартира Repo като процес и осигурява връзка с базата. Интерактивната работа се извършва чрез iex.bat -S mix, където потребителят може да извиква функциите от Logic. Данните се съхраняват физически във файла students.db, който е достъпен в папката на проекта. Миграциите позволяват лесно обновяване на структурата на таблицата без загуба на данни. Проектът демонстрира използване на Ecto за обвързване на структурирани данни с релационна база. Потребителят може да извършва добавяне, показване, изтриване и актуализиране на записи без директен достъп до SQL. Всички операции са типизирани и валидирани, което предотвратява въвеждането на некоректни данни. Крайният резултат е стабилен, структуриран и постоянен система за управление на студентски записи чрез база данни SQLite.

За реализиране на проекта са използвани:

1. Mix (системата за управление на проекти)

Mix е стандартният инструмент за управление на проекти в Elixir. Той позволява създаване на нови проекти, компилиране на код и стартиране на интерактивна среда. С Mix се управляват зависимостите чрез файла mix.exs, където се дефинират библиотеки и версии. Mix поддържа задачи като миграции на бази данни чрез Ecto и създаване на тестове. Чрез командата mix deps.get се изтеглят всички необходими зависимости от Hex и други репозитории. Mix осигурява възможност за стартиране на приложението в интерактивна среда чрез iex.bat -S mix.

Той подпомага автоматично компилиране на файлове при промяна в кода. Mix позволява създаване на персонализирани задачи, които могат да автоматизират различни операции. Инструментът е тясно интегриран с Ecto за работа с бази данни. Използването на Mix осигурява структуриран и подреден подход към разработката на Elixir приложения.

2. Ecto (библиотеката за работа с бази данни)

Ecto е официалната библиотека на Elixir за работа с бази данни и ORM функционалност. Тя позволява дефиниране на схеми, които съответстват на таблиците в базата данни. Ecto използва changeset за валидиране и безопасно обновяване на данни. Тя предоставя удобен синтаксис за създаване на заявки чрез Elixir кода вместо SQL. Ecto осигурява CRUD операции чрез функции като insert, update, delete и get_by. С помощта на Ecto се извършват миграции за създаване и промяна на структурата на таблиците. Тя поддържа множество бази данни чрез адаптери, като SQLite, PostgreSQL и MySQL. Ecto гарантира транзакционна сигурност и консистентност на данните. В комбинация с Mix, Ecto позволява автоматично стартиране на репозитории и миграции. Използването на Ecto улеснява разработката на базирани на данни приложения и предотвратява грешки при ръчно писане на SQL.

РЕАЛИЗАЦИЯ

1. Създаване на структурата на проекта

Създава се стандартна структура на Elixir проект, която разделя конфигурацията, логиката и базата данни. Папките lib, config и priv имат различни роли – код, настройки и миграции. Това позволява проектът да бъде подреден, мащабируем и лесен за поддръжка.

2. Добавяне на код към файловете

Във всеки файл се добавя конкретен код, който реализира връзката с базата данни чрез Ecto и SQLite. Repo управлява връзката с базата, Student описва структурата на данните, а Logic съдържа CRUD операциите. Конфигурацията задава къде се намира базата (students.db) и как приложението да работи с нея.

3. Стартиране и работа с приложението

Чрез терминала първо се инсталират зависимостите, след това се създава базата чрез миграция и накрая се стартира интерактивната среда. В iex се извикват функции за добавяне и извличане на студенти. Всички въведени данни се записват автоматично в SQLite файла students.db.

1. Създаване на структурата на проекта

Архитектурата включва

- mix.exs - конфигурация на проекта (зависимости и настройки)
- config.exs - за настройки
- Repo за връзка с базата от данни
- Application за стартиране на процеси
- Logic за бизнес логика
- Migration за структурата на базата от данни

student app/



Описание

<code>mix.exs</code>	конфигурация на проекта
<code>config/ config.exs</code>	настройки на приложението
<code>lib/</code>	основен код
<code>lib/student_app/</code>	домейн модули
<code>repo.ex</code>	връзка с база
<code>student.ex</code>	модел (схема)
<code>logic.ex</code>	бизнес логика
<code>lib/student_app.ex</code>	стартиране (OTP app)
<code>priv/</code>	вътрешни ресурси
<code>priv/repo/</code>	база данни
<code>migrations/</code>	промени в схема
<code>001_create_students.exs</code>	създаване таблица

2. Добавяне на код към файловете

`mix.exs`

```
defmodule StudentApp.MixProject do
  use Mix.Project

  def project do
    [
      app: :student_app,
      version: "0.1.0",
      elixir: "~> 1.15",
      start_permanent: Mix.env() == :prod,
      deps: deps()
    ]
  end
end
```

```

]
end
def application do
  [
    extra_applications: [:logger],
    mod: {StudentApp.Application, []}
  ]
end

defp deps do
  [
    {:ecto_sql, "~> 3.10"},
    {:ecto_sqlite3, "~> 0.10"}
  ]
end
end

```

Описание

defmodule StudentApp.MixProject do

Дефинира се модулът на проекта (всеки Elixir проект има такъв модул в `mix.exs`)

use Mix.Project

Включва функционалност от `Mix.Project`, това указва на Elixir, че този модул описва проект

def project do

Функция, която връща основните настройки на проекта

[

Започва keyword list (списък от настройки)

app: :student_app,

Име на приложението (атом) - използва се вътрешно от Mix/OTP

version: "0.1.0",

Версия на проекта

elixir: "~> 1.15",

Изисквана версия на Elixir обозначението `~>` означава „съвместима версия“ ($\geq 1.15, < 2.0$)

start permanent: Mix.env() == :prod,

Ако средата е **:prod**, приложението се стартира в permanent mode - ако crash е не - цялата система спира

deps: deps()

Взима зависимостите от функцията `deps/0`

]

end

Край на конфигурацията за проекта

def application do

Конфигурация за OTP приложението

```
[
```

Отново keyword list

```
extra_applications: [:logger],
```

Допълнителни приложения, които ще се стартират

:logger = вграден модул за логове

```
mod: {StudentApp.Application, []}
```

Основният модул на приложението (entry point)

{Module, args}, където се стартира supervision tree

```
]
```

```
end
```

Край на application конфигурацията

```
defp deps do
```

Частна функция (private) - дефинира зависимостите

```
[
```

Списък със зависимости

```
{:ecto_sql, "~> 3.10"},
```

Добавя библиотеката Ecto SQL за работа с бази данни

```
{:ecto_sqlite3, "~> 0.10"}
```

SQLite адаптер за Ecto, който позволява работа със SQLite база

```
]
```

```
end
```

Край на зависимостите

```
end
```

Край на модула

mix.exs - в обобщение:

- project/0 предназначен за общи настройки
- application/0 показва как се стартира приложението
- deps/0 работа с външни библиотеки

config/config.exs

```
import Config
```

```
config :student_app, StudentApp.Repo,  
  database: "students.db",  
  pool_size: 5,  
  journal_mode: :wal
```

```
config :student_app, ecto_repos: [StudentApp.Repo]
```

Описание

```
config/config.exs
import Config
```

Зарежда модула Config и позволява да се дефинират конфигурации

```
config :student_app, StudentApp.Repo,
```

Конфигурация за приложението :student_app, конкретно за модула StudentApp.Repo

```
  database: "students.db",
```

Име на SQLite базата данни

```
  pool_size: 5,
```

Брой едновременни връзки към базата

```
  journal_mode: :wal
```

SQLite режим (Write-Ahead Logging)- по-добра производителност

```
config :student_app, ecto_repos: [StudentApp.Repo]
```

Казва на Ecto кои са репозиторите (Repo) и списък с бази данни за приложението

```
lib/student_app/repo.ex
```

```
defmodule StudentApp.Repo do
  use Ecto.Repo,
    otp_app: :student_app,
    adapter: Ecto.Adapters.SQLite3
end
```

Описание

```
defmodule StudentApp.Repo do
```

Дефинира Repo модула (връзка с базата)

```
  use Ecto.Repo,
```

Включва функционалност от Ecto, дава CRUD операции (insert, update, delete)

```
  otp_app: :student_app,
```

Коя конфигурация да използва - взима настройки от config.exs

```
  adapter: Ecto.Adapters.SQLite3
```

Кой тип база данни използва - SQLite в случая

```
end
```

Край на модула

```
lib/student_app.ex
```

```
defmodule StudentApp.Application do
  use Application
```

```
  def start(_type, _args) do
    children = [
```

```

    StudentApp.Repo
  ]

  Supervisor.start_link(children, strategy: :one_for_one)
end
end

```

Описание

```
defmodule StudentApp.Application do
```

Основен OTP модул на приложението

```
  use Application
```

Позволява стартиране като OTP приложение

```
  def start( type,  args) do
```

Entry point при стартиране

```

    children = [
      StudentApp.Repo
    ]

```

Списък с процеси (children)- Repo се стартира като процес

```
    Supervisor.start_link(children, strategy: :one_for_one)
```

Стартира supervisor, ако процес падне - рестартира само него

```

  end
end

```

Край на модула

lib/student_app/logic.ex

```

defmodule StudentApp.Logic do
  import Ecto.Query
  alias StudentApp.{Repo, Student}

  def add_student(attrs) do
    %Student{}
    |> Student.changeset(attrs)
    |> Repo.insert()
  end

  def list_students do
    Repo.all(from s in Student, order_by: s.fnomer)
  end

  def delete_student(fnomer) do
    case Repo.get_by(Student, fnomer: fnomer) do
      nil -> {:error, "Not found"}
      student -> Repo.delete(student)
    end
  end
end

```



```

end
end

def update_student(fnomer, attrs) do
  case Repo.get_by(Student, fnomer: fnomer) do
    nil ->
      {:error, "Not found"}

    student ->
      student
      |> Student.changeset(attrs)
      |> Repo.update()
  end
end
end
end

```

Описание

```
defmodule StudentApp.Logic do
```

Модул с бизнес логика

```
import Ecto.Query
```

Позволява използване на заявки (from)

```
alias StudentApp.{Repo, Student}
```

Съкратени имена:

- Repo - StudentApp.Repo
- Student - StudentApp.Student

- add_student

```
def add_student(attrs) do
```

Функция за добавяне

```
%Student{}
```

Създава празна структура

```
|> Student.changeset(attrs)
```

Валидира и подготвя данни

```
|> Repo.insert()
```

Записва в базата

```
list_students
```

```
def list_students do
```

Взима всички студенти

```
Repo.all(from s in Student, order_by: s.fnomer)
```

SQL заявка чрез Ecto и сортиране по fnomer

- delete_student

```
def delete_student(fnomer) do
```

Изтрива по факултетен номер

```
case Repo.get by(Student, fnomer: fnomer) do
```

Търси студент

```
nil -> {:error, "Not found"}
```

Ако няма ивежда грешка

```
student -> Repo.delete(student)
```

Ако има изтрива

- update_student

```
def update_student(fnomer, attrs) do
```

Обновява студент

```
case Repo.get by(Student, fnomer: fnomer) do
```

Търси студент

```
nil ->  
{:error, "Not found"}
```

Ако няма извежда съобщение за грешка

```
student ->
```

Ако има го продължава

```
student  
|> Student.changeset(attrs)
```

Подготвя новите данни

```
|> Repo.update()
```

Записва промените

priv/repo/migrations/001 create students.exs

```
defmodule StudentApp.Repo.Migrations.CreateStudents do  
  use Ecto.Migration
```

```
  def change do  
    create table(:students) do  
      add :fname, :string  
      add :mname, :string  
      add :lname, :string  
      add :fnomer, :string  
      add :mark, :float  
    end
```

```
    create unique_index(:students, [:fnomer])  
  end  
end
```

Описание

```
defmodule StudentApp.Repo.Migrations.CreateStudents do
```

Миграция за създаване на таблица

```
  use Ecto.Migration
```

Включва DSL за база данни

```
  def change do
```

Основна функция за промени

```
    create table(:students) do
```

Създава таблица students

```
      add :fname, :string
```

Име

```
      add :mname, :string
```

Презиме

```
      add :lname, :string
```

Фамилия

```
      add :fnumber, :string
```

Факултетен номер

```
      add :mark, :float
```

Оценка

```
    end
```

Край на таблицата

```
    create unique index(:students, [:fnumber])
```

Уникален индекс - няма опция два студента да са с еднакъв номер

```
  end
```

```
end
```

Край на миграцията

3. Стартиране и работа с приложението

Указване на път до проекта

1. cd student app

```
PS C:\Users\vboji> cd C:\Users\vboji\Desktop\Elixir\Struct\Project\student_app
PS C:\Users\vboji\Desktop\Elixir\Struct\Project\student_app> dir
```

```
Directory: C:\Users\vboji\Desktop\Elixir\Struct\Project\student_app
```

Mode	LastWriteTime	Length	Name
d----	22.3.2026 г. 16:21		config
d----	22.3.2026 г. 16:17		lib
d----	22.3.2026 г. 16:17		priv
-a----	22.3.2026 г. 16:22	470	mix.exs

2. mix deps.get

```

PS C:\Users\vboji\Desktop\Elixir\Struct\Project\student_app> mix deps.get
Resolving Hex dependencies...
Resolution completed in 0.188s
New:
  cc_precompiler 0.1.11
  db_connection 2.9.0
  decimal 2.3.0
  ecto 3.13.5
  ecto_sql 3.13.5
  ecto_sqlite3 0.22.0
  elixir_make 0.9.0
  exqlite 0.35.0
  telemetry 1.4.1
* Getting ecto_sql (Hex package)
* Getting ecto_sqlite3 (Hex package)
* Getting decimal (Hex package)
* Getting ecto (Hex package)
* Getting exqlite (Hex package)
* Getting cc_precompiler (Hex package)
* Getting db_connection (Hex package)
* Getting elixir_make (Hex package)
* Getting telemetry (Hex package)
* creating c:/Users/vboji/.mix/elixir/1-19-otp-28/rebar3

```

3. mix ecto.migrate

```

PS C:\Users\vboji\Desktop\Elixir\Struct\Project\student_app> mix ecto.migrate
Generated elixir_make app
==> decimal
Compiling 4 files (.ex)
Generated decimal app
==> student_app
==> Analyzing applications...
==> Compiling telemetry
==> ecto
Compiling 56 files (.ex)
Generated ecto app
==> db_connection
Compiling 18 files (.ex)
Generated db_connection app
==> ecto_sql
Compiling 25 files (.ex)
Generated ecto_sql app
==> cc_precompiler
Compiling 3 files (.ex)
Generated cc_precompiler app
==> exqlite
Downloading precompiled NIF to c:/Users/vboji/AppData/Local/elixir_make/cache/exqlite-nif-2.17-x86_64-windows-msvc-0.35.0.tar.gz
Compiling 12 files (.ex)
Generated exqlite app
==> ecto_sqlite3
Compiling 5 files (.ex)
Generated ecto_sqlite3 app
==> student_app
Compiling 4 files (.ex)
Generated student_app app

16:41:35.477 [info] == Running 1 StudentApp.Repo.Migrations.CreateStudents.change/0 forward
16:41:35.481 [info] create table students
16:41:35.482 [info] create index students_fnumber_index
16:41:35.487 [info] == Migrated 1 in 0.0s

```

4. iex.bat -S mix

```

PS C:\Users\vboji\Desktop\Elixir\Struct\Project\student_app> iex.bat -S mix
Erlang/OTP 28 [erts-16.2] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit:ns]

Interactive Elixir (1.19.5) - press Ctrl+C to exit (type h() ENTER for help)

```

5. Записваме в интерактивен режим (ОТКОПИРА СЕ ВСИЧКО НАВЕДНЪЖ)

```
alias StudentApp.Logic
```

```

Logic.add_student(%{
  fname: "Ana",
  mname: "Antonova",
  lname: "Antonova",
  fnumber: "10000000",
  mark: 4.0
})

```

```
})
```

Logic.list_students()

```
PS C:\Users\vboji\Desktop\Elixir\Struct\Project\student_app> iex.bat -S mix
Erlang/OTP 28 [erts-16.2] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit:ns]

Interactive Elixir (1.19.5) - press Ctrl+C to exit (type h() ENTER for help)
iex(1)> alias StudentApp.Logic
StudentApp.Logic
iex(2)>
nil
iex(3)> Logic.add_student(%{
    fname: "Ana",
    mname: "Antonova",
    lname: "Antonova",
    fnumber: "10000000",
    mark: 4.0
})

21:08:51.062 [debug] QUERY OK source="students" db=2.2ms decode=1.9ms idle=1081.3ms
INSERT INTO "students" ("mark","fname","mname","lname","fnumber") VALUES (?1,?2,?3,?4,?5) RETURNING "id"
[4.0, "Ana", "Antonova", "Antonova", "10000000"]
{:ok,
 %StudentApp.Student{
   __meta__: #Ecto.Schema.Metadata<:loaded, "students">,
   id: 5,
   fname: "Ana",
   mname: "Antonova",
   lname: "Antonova",
   fnumber: "10000000",
   mark: 4.0
 }}
iex(4)>
nil
```

Данните се записват в БД

Актуализация на запис

```
alias StudentApp.Logic
```

```
Logic.update_student("00000000", %{
  fname: "Anna",
  mark: 5.90
})
```

```
StudentApp.Logic
iex(15)>
nil
iex(16)> Logic.update_student("10000000", %{
    fname: "Anna",
    mark: 5.90
})

21:28:11.726 [debug] QUERY OK source="students" db=0.3ms idle=159.8ms
SELECT s0."id", s0."fname", s0."mname", s0."lname", s0."fnumber", s0."mark" FROM "students" AS s0 WHERE (s0."fnumber" = ?) ["10000000"]
]

21:28:11.727 [debug] QUERY OK source="students" db=0.2ms idle=160.2ms
UPDATE "students" SET "mark" = ?, "fname" = ? WHERE "id" = ? [5.9, "Anna", 6]
{:ok,
 %StudentApp.Student{
   __meta__: #Ecto.Schema.Metadata<:loaded, "students">,
   id: 6,
   fname: "Anna",
   mname: "Antonova",
   lname: "Antonova",
   fnumber: "10000000",
   mark: 5.9
 }}
iex(17)> Logic.list_students()
```

Проверка след актуализация

Logic.list_students()

```
iex(17)> Logic.list_students()

21:28:19.035 [debug] QUERY OK source="students" db=0.2ms queue=0.1ms idle=1431.3ms
SELECT s0."id", s0."fname", s0."mname", s0."lname", s0."fnumber", s0."mark" FROM "students" AS s0 ORDER BY s0."fnumber" []
[
  %StudentApp.Student{
    __meta__: #Ecto.Schema.Metadata<:loaded, "students">,
    id: 6,
    fname: "Anna",
    mname: "Antonova",
    lname: "Antonova",
    fnumber: "10000000",
    mark: 5.9
  }
]
```

ако няма такъв fnumber ще получим:

{:error, "Not found"}

```
iex(18)> alias StudentApp.Logic
StudentApp.Logic
iex(19)> Logic.update_student("00000000", %{
  fname: "Anna",
  mark: 5.90
})

21:31:18.440 [debug] QUERY OK source="students" db=0.3ms idle=1463.5ms
SELECT s0."id", s0."fname", s0."mname", s0."lname", s0."fnumber", s0."mark" FROM "students" AS s0 WHERE (s0."fnumber" = ?) ["00000000"]
[
{:error, "Not found"}]
```

Изтриване на запис

alias StudentApp.Logic

Logic.delete_student("10000000")

```
iex(21)> Logic.delete_student("10000000")

21:32:19.991 [debug] QUERY OK source="students" db=0.3ms idle=479.9ms
SELECT s0."id", s0."fname", s0."mname", s0."lname", s0."fnumber", s0."mark" FROM "students" AS s0 WHERE (s0."fnumber" = ?) ["10000000"]
[

21:32:19.992 [debug] QUERY OK source="students" db=0.4ms idle=480.3ms
DELETE FROM "students" WHERE "id" = ? [6]
{:ok,
 %StudentApp.Student{
   __meta__: #Ecto.Schema.Metadata<:deleted, "students">,
   id: 6,
   fname: "Anna",
   mname: "Antonova",
   lname: "Antonova",
   fnumber: "10000000",
   mark: 5.9
 }}]
```

Проверка след изтриване

Logic.list_students()

```
iex(22)> Logic.list_students()

21:32:44.521 [debug] QUERY OK source="students" db=0.3ms idle=803.8ms
SELECT s0."id", s0."fname", s0."mname", s0."lname", s0."fnumber", s0."mark" FROM "students" AS s0 ORDER BY s0."fnumber" []
[]
```